

DeepSeek-R1

Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

1. 개요

문제

최근 대규모 언어모델은 수학, 논리, 코딩처럼 복잡한 추론이 필요한 문제에서도 높은 성능을 보임.

특히 **사고 과정(CoT)**을 출력하게 하면 문제를 한 번에 풀 때보다 성능이 좋아지는 경우가 많음.

기존 추론모델은 대부분 사람이 만든 고품질 추론 과정을 대량으로 학습하는 구조

문제
↓
사람이 작성한 단계별 풀이
↓
모델이 해당 풀이 방식 학습
↓
비슷한 방식으로 문제 해결

but...

- 좋은 추론 데이터를 사람이 직접 만드는 데 많은 비용 필요
- 사람이 만든 풀이가 항상 가장 좋은 풀이 방법이라고 볼 수 없음
- 모델이 사람의 사고방식만 따라가면서 새로운 추론 전략을 찾기 어려울 수 있음

그럼 아예 풀이 방법을 알려주지 않고 정답을 맞혔는지만 알려주면 모델이 스스로 좋은 추론 방법을 찾을 수 있지 않을까?

DeepSeek

사람이 만든 추론 데이터를 먼저 학습시키지 않고 강화학습만으로 추론 능력을 향상시키는 실험을 진행

1) DeepSeek-R1-Zero

DeepSeek-V3-Base → 강화학습 → DeepSeek-R1-Zero

사람이 다시 확인해라, 틀렸으면 돌아가라, 다른 방법을 찾아라 같은 추론 방법을 따로 가르치지 않고

대신 최종 정답을 맞혔는지를 중심으로 보상 제공

⇒ 학습이 진행될수록 모델이 스스로 자기검토, 오류 수정, 재시도, 풀이 되돌아가기, 다른 풀이 탐색같은 추론 행동을 더 많이 사용하게 됨

but... R1-Zero는 추론 능력은 매우 높아졌지만 실제 사용에는 문제가 있었음.

- 답변이 지나치게 길고 읽기 어려움
- 영어와 중국어가 섞임
- 글쓰기나 일반 질문에는 상대적으로 약함

2) DeepSeek-R1

이를 개선한 모델, 강화학습만 사용하는 것이 아니라 지도 미세조정 + 강화학습을 여러 단계로 결합한 구조

⇒ 충분히 강한 기반모델에 어려운 문제와 정확한 보상을 제공하면 사람이 추론 과정을 직접 가르치지 않아도 복잡한 추론 행동을 강화할 수 있음

2. 기존 연구

2.1. 기존 추론모델

기존 대규모 언어모델의 후처리 학습은 일반적으로 지도 미세조정(SFT) → 강화학습(RL) 순서

지도 미세조정

사람이 만든 정답이나 좋은 추론 과정을 그대로 모델에게 보여주고 학습시키는 방법

장점

- 학습이 안정적임
- 원하는 답변 형식을 쉽게 만들 수 있음
- 사람이 보기 좋은 풀이를 만들기 쉬움

단점

사람이 제공한 풀이가 모델에게 최적의 풀이 방법이라는 보장은 없음.

특히 사람이 작성한 정답에서는 중간 오류, 다시 생각하는 과정, 여러 풀이 비교, 실패 후 재 시도 같은 과정이 대부분 생략

= 사람이 만든 풀이만 학습하면 스스로 시행착오를 통해 더 좋은 풀이 방식을 찾을 기회가 줄어들음

2.2. 기존 강화학습

강화학습: 모델이 만든 답변을 평가한 뒤 좋은 답변을 더 자주 생성하도록 학습하는 방식

문제 → 모델 답변 생성 → 답변 평가 → 보상 → 좋은 답변의 생성 확률 증가

기존 대규모 언어모델 강화학습에서는 **PPO**라는 방법이 많이 사용됨.

PPO는 일반적으로

- 정책모델: 실제 답변 생성
- 기준모델: 모델이 너무 크게 변하지 않도록 기준 제공
- 보상모델: 답변 평가
- 가치모델: 앞으로 받을 보상 예측

등이 필요

여기서 DeepSeek이 특히 문제라고 본 것이 **가치모델!!**

왜?

가치모델은 현재까지 생성된 내용을 보고 **이 답변이 마지막에 얼마나 좋은 결과를 낼까?** 를 예측해야 함

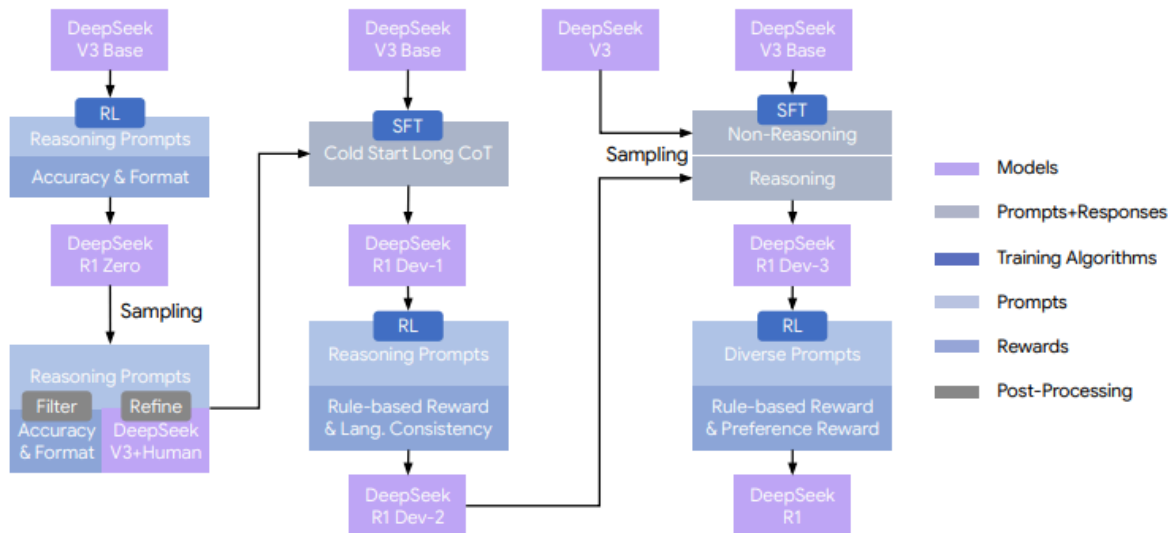
근데 앞부분만 보면 틀릴 것 같은 답변도 뒤에서 스스로 수정해서 정답을 맞힐 수 있기 때문에

긴 추론에서는 **중간 상태만 보고 최종 보상을 예측하는 것이 어려움.**

+ 가치모델 자체도 큰 신경망이기 때문에 계산량과 메모리가 많이 필요

그래서 DeepSeek은 가치모델을 없애는 **그룹 상대 정책 최적화(GRPO)**를 사용!

3. DeepSeek-R1-Zero



3.1. 그룹 상대 정책 최적화(GRPO)

가치모델 제거하고 같은 문제에 대해 생성된 여러 답변을 서로 비교하자

작동 방식

- 하나의 질문 q 에 대해 여러 답변을 생성
 $\{o_1, o_2, \dots, o_G\}$: 논문에서는 한 문제당 **16개의 답변** 생성
- 각 답변에 대한 보상을 $r_1, r_2, \dots, r_G$ 라고 설정

답변	결과	보상
답변 1	정답	1
답변 2	오답	0
답변 3	정답	1
답변 4	오답	0

- 각 답변의 상대적 성과 계산

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

A_i : 해당 답변이 같은 문제에 대한 다른 답변보다 얼마나 좋은지를 나타냄

- 평균보다 높은 보상 ($A_i > 0$) → 해당 답변을 생성할 확률을 높임
- 평균보다 낮은 보상 ($A_i < 0$) → 해당 답변을 생성할 확률을 낮춤

= 이 답변의 절대적 가치가 얼마인지 별도의 모델이 예측하는 대신 같은 문제 답변들끼리 상대평가

4. 모델이 한 번에 너무 크게 변하는 것을 막기 위해 기존 모델과의 차이를 제한하는 항 함께 사용

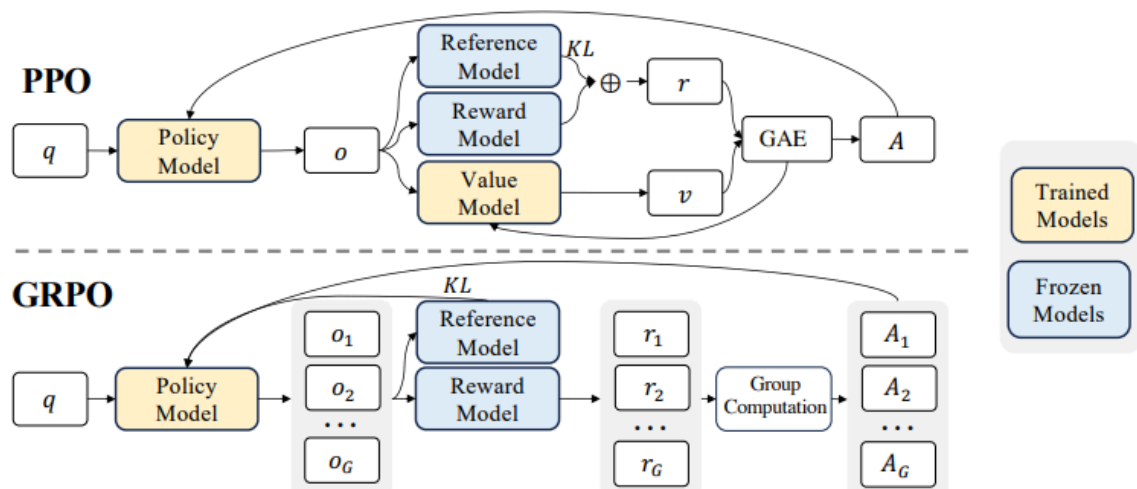
목표 = 좋은 답변의 생성확률 증가 - 기존 모델과의 지나친 차이목표

전체 목적함수

$$J_{GRPO}(\theta) = E \left[\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta D_{KL} \right) \right]$$

- **A_i** : 같은 문제의 답변 중 어떤 답변이 더 좋았는지
- **clip**: 모델이 한 번의 학습으로 너무 크게 변하지 않도록 제한
- D_{KL} : 기존 모델과 현재 모델의 차이가 지나치게 커지는 것을 방지

⇒ 좋은 답변의 확률은 높이고 모델이 갑자기 이상하게 변하는 것은 막는 구조



PPO	GRPO
가치모델 필요	가치모델 없음

PPO	GRPO
앞으로 받을 보상 예측	같은 문제 답변끼리 비교
긴 추론에서 가치 예측 어려움	최종 결과를 이용해 상대평가
계산량 + 메모리 큼	상대적으로 효율적

** 논문 실험에서는 PPO도 세부 설정을 잘 조정하면 GRPO에 가까운 성능을 낼 수 있었지만

PPO는 가치모델을 추가로 학습해야 하고 세부 설정에도 민감해서 **대규모 추론모델 학습에는 GRPO가 더 실용적**

3.2. 보상 설계

R1-Zero: 복잡한 신경망 기반 보상모델보다 규칙 기반 보상을 사용

$$Reward_{rule} = Reward_{accuracy} + Reward_{format}$$

1) 정답 보상

모델이 최종적으로 정답을 맞혔는지 평가

- 수학 문제 → 기준 정답과 비교
- 코딩 문제 → 프로그램을 실행하여 시험 사례 통과 여부 확인
- 논리 문제 → 정답 선택지 또는 숫자와 비교

중간 추론 과정이 사람처럼 좋은지를 직접 평가하지 않음

ex) 수학 문제에서 풀이 과정이 다소 돌아가더라도 최종 정답이 맞으면 보상을 받을 수 있음

2) 형식 보상

모델이 추론 과정과 최종 답을 정해진 형식에 맞게 구분하도록 보상



왜 보상모델을 쓰지 않았나?

사람처럼 자연스럽게 추론했는지 보상모델이 평가하면 더 좋지 않냐는 의문이 있는데

쓰지 않은 가장 큰 이유는 **보상 해킹** 때문임

보상 해킹: 모델이 실제 문제를 잘 푸는 게 아니라 평가모델한테 높은 점수를 받는 방법을 배우는 현상

ex) 보상모델이 긴 답변 = 좋은 답변이라고 판단하는 경향이 있다면 모델은 정답을 잘 찾는 대신 답변만 길게 쓰는 방법을 학습할 수 있음

실제로 강화학습을 오래 진행했을 때 보상 점수는 계속 상승하고 실제 Codeforces 성능은 감소함

⇒ **보상값이 높다고 실제 성능도 좋아진다고 볼 수 없음**

그래서 정답을 확실하게 확인할 수 있는 수학·코딩 문제에서는 최대한 객관적인 규칙 기반 보상을 사용

3.3. R1-Zero 학습

기본모델: **DeepSeek-V3-Base**

- 전체 약 671B 매개변수
- 한 토큰 처리 시 약 37B 매개변수 활성화
- 약 14.8조 토큰으로 사전학습

= 아무것도 모르는 모델에게 강화학습을 해서 추론 능력을 처음 만들어낸 건 아님

모델 안에 이미 계산, 풀이, 오류 수정, 여러 형태의 추론과 관련된 패턴이 어느 정도 존재했을 가능성이 높아서 강화학습이 추론 능력을 완전히 0에서 생성했다기보다는 **사전학습으로 이미 존재하던 여러 추론 패턴 중 정답에 도움이 되는 행동을 선택하고 강화했다고** 봐야됨

3.4. 자가 발전

강화학습을 진행하면서 모델의 답변 길이가 점점 증가

= 초기에는 비교적 짧게 답했지만 학습 후반에는 문제를 더 길게 검토하고 여러 방법을 시도하기 시작함.

특히 다시 계산하기, 앞의 결과 확인, 오류 찾기, 풀이 되돌리기, 새로운 방법 시도하기 등의 행동이 나타남.

Aha moment

모델이 수학 문제를 풀다가 **Wait, wait. Wait.** 라고 한 뒤 기존 계산을 다시 확인 → 이후 기존 풀이가 잘못되었다고 판단하고 다른 방향으로 다시 문제를 풀기 시작한 사례가 있었음

DeepSeek은 이를 모델의 **자가 발전 과정에서 나타난 흥미로운 변화**로 설명!

실제로 자기검토가 증가했나?

연구진이 학습 과정에서 wait, mistake, however, retry, error, verify, wrong, check같은 자기검토 관련 단어를 직접 확인했을 때 → 강화학습이 진행될수록 해당 표현의 빈도가 약 5~7배 증가

⇒ 모델이 단순히 답변을 길게 만든 것뿐 아니라 **이전 내용을 확인하고 수정하는 형태의 추론 패턴도 증가**

근데 모델이 의식적으로 반성했다기보다 **정답률을 높이는 데 유리한 자기검토 형태의 출력이 강화된 것**

3.6. R1-Zero의 문제

추론 문제에서는 높은 성능을 보였지만 실제 사용자 입장에서는 불편함

1. 가독성: 추론이 지나치게 길어지고 중간 내용도 정리되지 않는 경우 발생
2. 언어 혼용:

```
Let's solve this problem.  
首先我们可以...  
Then we calculate...
```

처럼 영어와 중국어가 섞임.

3. 일반 능력 부족: 강화학습 대부분이 수학, 코딩, 논리, 과학·공학처럼 정답을 확인하기 쉬운 문제를 중심으로 이루어져서
글쓰기, 일반적인 질문, 사용자 지시 수행에서는 상대적으로 성능이 낮음

⇒ 최종 DeepSeek-R1에서는 **지도 미세조정을 다시 추가**

4. DeepSeek-R1

DeepSeek-V3-Base

↓

초기 지도 미세조정

↓

1차 추론 강화학습

↓

정답 응답 선별

↓

대규모 지도 미세조정

↓

2차 강화학습

↓

DeepSeek-R1

R1-Zero: 강화학습만으로 추론 능력이 얼마나 발전하는지 확인하는 실험적 모델

R1: R1-Zero의 추론 능력을 살리면서 실제 사용성까지 개선한 최종 모델

4.1. 초기 지도 미세조정

R1-Zero가 생성한 답변 중 좋은 추론 결과를 가져옴.

선별 기준

- 최종 답이 맞음
- 형식이 정상적임
- 반복이 심하지 않음
- 언어 혼용이 적음

이후 사람과 DeepSeek-V3를 이용해 풀이를 읽기 좋은 형태로 수정

R1-Zero의 원래 추론
↓
정답인 답변만 선택
↓
반복 / 언어 혼용 제거
↓
사람과 DeepSeek-V3가 표현 정리
↓
초기 학습 데이터



왜 굳이 지도 미세조정을 다시 넣음?

R1-Zero에서 확인했듯이 강화학습만 하면 추론 성능은 좋아질 수 있지만 결과물이 사람이 읽기 좋은 형태라는 보장은 없음.

그래서 초기 지도 미세조정은 추론 방법 자체를 처음부터 가르치는 것보다는 이미 발견한 추론 패턴을 사람이 읽기 좋은 형태로 정리하는 역할에 가까움.

4.2. 1차 추론 강화학습

초기 지도 미세조정 이후 다시 강화학습 진행 → 기존 정답 보상에 언어 일관성 보상 추가
(R1-Zero에서는 영어와 중국어가 섞이는 문제가 있었기 때문)

언어 일관성 보상

$$Reward_{language} = \frac{Num(Words_{target})}{Num(Words)}$$

- $Words_{target}$: 질문과 같은 언어의 단어
- Words: 전체 단어

결과

언어 혼용은 줄어들었지만 일부 코딩 문제의 성능은 조금 감소함

⇒ 모델의 순수 문제 해결 성능만 극대화하는 것과 사람이 읽기 좋은 출력으로 만드는 것이 항상 같은 방향은 아님

즉 성능과 사용자 경험 사이에 어느 정도 절충 필요

4.3. 정답 응답 선별

1차 강화학습 모델에게 한 문제당 여러 답을 생성하게 하고 정답인 답변만 남겨 다시 지도 미세조정 데이터로 사용

문제

답변 A → 오답 → 제거
답변 B → 정답 → 사용
답변 C → 정답 → 사용
답변 D → 오답 → 제거

이를 통해 강화학습 모델이 직접 만든 좋은 추론 과정을 다시 학습 데이터로 활용

4.4. 약 80만 개 지도학습 데이터

최종적으로 약 **80만 개**의 데이터를 이용해 지도 미세조정을 진행

추론 데이터

약 60만 개

분야	데이터 수
수학	약 39.5만
코드	약 21.1만
과학·공학	약 1만
논리	약 1만

일반 데이터

약 20만 개

- 글쓰기

- 사실 질의응답
- 번역
- 역할 수행
- 일반 대화
- 소프트웨어 개발 등 포함

왜 일반 데이터를 같이 넣음?

R1-Zero처럼 추론 데이터만 학습하면 수학은 잘 푸는데 일반적인 대화는 이상한 모델이 될 수 있기 때문

따라서 추론 성능을 유지하면서 **일반적인 언어모델 능력도 함께 복구하기 위해**

4.5. 2차 강화학습

1) 문제

추론 문제: 수학·코딩처럼 객관적인 정답이 존재 → 규칙 기반 보상 사용

일반 문제: 글쓰기나 일반 질문처럼 정답이 하나가 아님 → 보상모델 사용

2) 보상

$$Reward = Reward_{reasoning} + Reward_{general} + Reward_{language}$$

일반 보상

- **유용성:** 사용자의 질문에 실제로 도움이 되는 답인지 평가
- **안전성:** 위험하거나 부적절한 내용을 생성하는지 평가

5. 실험 결과

전체적으로 가장 큰 성능 향상은 **복잡한 추론이 필요한 수학과 코딩 문제**에서 나타남.

반면 단순 사실 질의응답이나 실제 소프트웨어 개발 문제 등에서는 모든 다른 모델보다 항상 우수한 것은 아니었음.

= DeepSeek-R1의 강점은 **긴 추론과 검증이 필요한 문제에서 특히 크게 나타남** + 문제의 난이도가 높아질수록 모델이 더 많은 추론 토큰을 사용하는 특성

6. 지식 종류

DeepSeek-R1은 매우 큰 모델이기 때문에 학습과 사용에 많은 계산 자원이 필요

→ 이를 위해 R1이 만든 고품질 추론 데이터를 작은 모델에 학습시키는 **지식 증류**를 진행

⇒ **작은 모델에 직접 대규모 강화학습을 하는 것보다 큰 R1 모델의 추론 결과를 학습시키는 것이 더 좋은 성능을 보임**

즉 새로운 추론 전략을 발견하는 것은 큰 모델이 담당하고 작은 모델은 그 결과를 학습하는 것이 더 효율적일 수 있다는 것을 보여줌.

7. 한계 및 결론

- **도구 활용 부족:** 검색엔진, 계산기와 같은 외부 도구 사용 능력이 부족
- **추론 효율 문제:** 쉬운 문제에서도 필요 이상으로 오래 생각하는 과잉추론이 발생할 수 있음
- **언어 혼용:** 중국어와 영어 중심으로 학습되어 다른 언어에서 언어가 섞일 가능성이 있음
- **문장 지시에 민감:** 여러 예시를 제공하는 방식이 오히려 성능을 낮추는 경우가 있음
- **실제 소프트웨어 개발 능력:** 평가 비용이 높아 대규모 강화학습을 충분히 적용하지 못함
- **보상 해킹:** 신뢰할 수 있는 정답 검증기가 없는 문제에서는 보상모델의 허점을 이용할 위험이 있음

가장 근본적인 한계는 **정답을 객관적으로 검증할 수 있는가**임

수학이나 코딩은 정확한 보상을 주기 쉽지만 좋은 글쓰기, 창의성, 복잡한 의사결정, 가치판단같이 하나의 정답이 없는 문제는 좋은 보상을 정의하기 어려움

따라서 최종 DeepSeek-R1은 강화학습으로 추론 능력을 발전시키고, 지도 미세조정으로 가독성과 일반적인 언어 능력을 보완하는 다단계 학습 방식을 제시